WNDRVR
SUPPORT NETWORK

VXWORKS 7 SDK GENERATOR PLATFORM DEVELOPER GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 SDK Generator Platform Developer Guide, SR0650

10 July 2020

1. ABOUT THE PLATFORM DEVELOPER

The role of the *Platform Developer* is to create and manage VxWorks® VSB and VIP projects to produce an SDK for the *Application Developer*. As well, they can integrate new applications or libraries (produced by the application developer) into a VxWorks project.

SDK (Software Development Kit)

A complete system that is ready for application development. The application developer does not need to know about setting up VxWorks platform projects.

VSB (VxWorks Source Build)

A VSB is a VxWorks platform project that allows you to select and compile VxWorks kernel libraries with selected features for a specific CPU architecture or a Board Support Package (BSP) to create a kernel archive.

VIP (VxWorks Image Project)

A VIP is a VxWorks platform project that allows you to configure which libraries to use from the kernel archive to produce a VxWorks kernel image that can be booted on the target platform. To create a VIP, you must first create and build a VSB project.

2. SDK PREPARATION CONSIDERATIONS

Prior to generating an SDK, a platform developer must consider how the application developers will use an SDK so that they have everything they need.

SDK Characteristic	Choices	Notes
Host type	Windows 10 Ubuntu 18.04 LTS Ubuntu 20.04 LTS	If another host type is required, contact your Wind River representative.
Target architecture	ARM PowerPC RISC-V X86	Each SDK is created based on a specific architecture. Pre-defined configurations are available for specific architectures and associated hardware. **ARM (fsl_imx6, ti_sitara_a8, rpi_4)** **PowerPC (fsl_t1)** **RISC-V (sifive_generic)** **X86 (itl_generic, CORE CPU)**
Run-time environment	Hardware target QEMU emulator	Setting up and accessing the hardware is beyond the scope of this documentation.
Debugging	wrdbg Simics	The Wind River Debugger (wrdbg) requires specific configuration settings in the VSB/VIP. Other debugging methods are beyond the scope of this documentation. For information about purchasing Simics, contact your Wind River representative. To include files for other debuggers (such as Simics scripts) use the --add option of the SDK generator.
Third party add-ons	CMake Boost Python Rust	VxWorks supports a number of third-party add-ons that can be used for application development in an SDK. To include sample files for any of these, use the --add option of the SDK generator. Pre-defined configurations are available for specific architectures and associated hardware. For more information, see below this table.
Application type	Kernel applications (DKMs) User applications (RTPs)	An SDK can support either or both: **Downloadable kernel modules (DKMs)** **Real-time processes (RTPs)**
VSB/VIP preparation	Use standard VSB/VIP projects (recommended) Use vxbuild.py script (use only at your discretion)	To use the standard VSB/VIP projects, refer to the standard VxWorks 7 documentation. The vxbuild.py script requires a JSON configuration specification. Examples are provided in the VxWorks 7 *installDir* **/sdk-generator/configs**.

SDK Characteristic	Choices	Notes
Documentation	Kernel API References Application API References	By default, the generated SDK does not include VxWorks reference documentation. Kernel reference material is used with DKMs. Application reference material is used with RTPs.

CMake

CMake is an open-source, cross-platform family of tools which can be used with the SDK to build, test and package software.

By default, a generated VxWorks SDK includes CMake. When the application developer sources the SDK environment, they will automatically have access to CMake. There is no need to download, install and configure Cmake separately. With VxWorks CMake, you can build existing CMake examples that are included in the generated SDK, or you can create your own projects.

Boost

A Boost-enabled SDK provides the application developer with the necessary layers/components to build and run Boost applications on a VxWorks target. For more information no how to enable Boost within your VSB/VIP, see:

VxWorks 7 Third-Party Software Support For Boost

The following pre-defined configuration files are available:

- ARM_BOOST.json (fsl_imx6)
- PPC_BOOST.json (fsl_t1)
- x86_BOOST.json (itl_generic)

Python

A Python-enabled SDK provides the application developer with the necessary layers/components to run Python applications on a VxWorks target. For more information no how to enable Python within your VSB/VIP, see:

VxWorks 7 Third-Party Software Support For Python

The following pre-defined configuration files are available:

- RPI_4_PYTHON.json (ARM rpi_4)
- x86_PYTHON.json (itl_generic)

Rust

A Rust-enabled SDK provides the Application Developer with the necessary layers/components to build and run Rust applications on a VxWorks target. For more information no how to enable Rust within your VSB/VIP, see:

VxWorks 7 Third-Party Software Support For Rust

The following pre-defined configuration files are available:

- ARM_RUST.json (fsl_imx6)
- x86_RUST.json (itl_generic)

3. PREPARING A VXWORKS IMAGE WITH VXBUILD.PY

The vxbuild.py script allows a platform developer to create a VSB and VIP using JSON configuration files.

About This Task

Note: This capability is only for use at your discretion. Your installation includes sample configuration files for various target architectures and application add-ons that Wind River used during testing. However, you must ensure that configurations are adjusted to match your target.

Before You Begin

- Ensure you have considered what all application developers will need for building (compiling) their code, debugging, and testing; see SDK Preparation Considerations on page 2.
- Install VxWorks 7 SR0650 (note the path to the installation directory; that is, *installDir*).
- Determine the version of python you need. You need at least version 3.6 for the SDK generator.
  - If the default on your host is version 2*x*, use the python3 command.
  - VxWorks SR0650 includes version 3.8. Using the wrenv script allows access to this version.

Procedure

1. Create or navigate to your workspace directory; that is, *workspaceDir*.
2. Create a JSON configuration file or use one of the pre-defined files in *installDir***/sdk-generator/configs**.

   Configuration files are provided for:

   - Each of the supported architectures: ARM, PPC, RISC-V, and X86 (for example, **x86.json**, which has been used by Wind River only with a QEMU target).
   - Each of the supported add-ons: Python, Rust, and Boost (for example, **x86_PYTHON.json**).

   The configuration files for the add-ons have additional data compared to the parent configuration that allows the SDK to build additional application types.

3. Run the wrenv script within the VxWorks installation directory.

   This allows the -i *INSTALL_DIR* option to be omitted.

Host Type	Instructions
Linux	`$ cd installDir` `$ ./wrenv.sh -p vxworks-7`
Windows	`$ cd installDir` `$ wrenv.exe -p vxworks-7`

4. If required, invoke the script using the -h option to see the syntax.

Host Type	Instructions
Linux	`$ installDir /sdk-generator/vx_build/vxbuild.py -h`
Windows	`$ installDir \sdk-generator\vx_build\vxbuild.py -h`

```
usage: vxbuild.py [-h] [-i INSTALL_DIR] -c CONFIG -ws WORKSPACE
                  [-o PROJECT_NAME] [-f] [-v]

optional arguments:
  -h, --help            show this help message and exit
  -i INSTALL_DIR, --install INSTALL_DIR
                        Path to VxWorks 7 installation directory
  -c CONFIG, --config CONFIG
                        Path to JSON config file
  -ws WORKSPACE, --workspace WORKSPACE
                        Output folder for generated VxWorks projects
  -o PROJECT_NAME       Project name for generated VxWorks projects
  -f, --force           Overwrite the existing output directory
  -v                    Output additional information to the console
```

5. Run the vxbuild.py script to create a VSB, VIP, and the kernel image.

   For example, using the pre-defined **x86.json** file from the *workspaceDir*:

Host Type	Instructions
Linux	`$ installDir /sdk-generator/vx_build/vxbuild.py -c installDir /sdk-generator/configs/x86.json -ws . -o x86-predefine`
Windows	`$ installDir \sdk-generator\vx_build\vxbuild.py -c installDir \sdk-generator\configs\x86.json -ws workspaceDir -o x86-predefine`

   This creates and builds:

   - A VSB project, **x86-predefine_VSB**.
   - A VIP project, **x86-predefine_VIP**. The kernel image is:
     workspaceDir **/x86-predefine_VIP/default/vxWorks**

6. It is recommended to load the kernel image on the target and do some basic sanity testing.

4. GENERATING AN SDK

As a platform developer, use the SDK Generator tool to generate a custom SDK for an application developer.

Before You Begin

- Ensure you have considered what all application developers will need for building (compiling) their code, debugging, and testing; see SDK Preparation Considerations on page 2.
- Install VxWorks 7 SR0650 (note the path to the installation directory; that is, *installDir*).
- Determine the version of python you need. You need at least version 3.6 for the SDK generator.

  Note: Ensure that you have the proper version of Python set as the default for accessing **.py** files.

  - If the default on your host is version 2*x*, use the python3 command.
  - VxWorks SR0650 includes version 3.8. Using the *installDir* **/wrenv.sh** allows access to this version.
- If required, install QEMU; see *VxWorks 7 SDK QEMU Installation Guide*.

Procedure

1. If using the vxbuild.py script to create a kernel image, see Preparing a VxWorks Image with vxbuild.py on page 4 and continue with Step 4.
2. Create and build a VxWorks Source Build (VSB) project with all layers and options enabled for the functionality that the application developers need.

   Use the standard VxWorks 7 project workflows.

   Building the VSB produces the libraries you will need to create a VxWorks image for your board.

3. Create and build a VxWorks image project (VIP) with all components and options enabled for the functionality that the application developers need.

   Some key components include the debug agent, the drivers for a serial port and an Ethernet port, the VxWorks kernel target and command shells, and so on. Typically, using **PROFILE_DEVELOPMENT** when you create the VIP will include these. Other recommended components are:

   - **INCLUDE_DEBUG_AGENT_START** (automatically starts up the TCF/Debug Agent needed for wrdbg access)
   - **INCLUDE_IFCONFIG** (allows the ifconfig utility)
   - **INCLUDE_PING** (allows the ping utility)
   - **INCLUDE_ROUTECMD** (allows the routec utility)

   Building the VIP produces a VxWorks kernel image. The SDK uses the kernel image as the basis for developing the applications.

4. If required, install or access at least version 3.6 of python.

Host Type	Instructions
Linux	To install directly on the host: `$ sudo apt-get install python3 && python3 -m pip install -U pip`

Host Type	Instructions
	Or, to use the version included with your VxWorks 7 installation: $ **cd** *installDir* $ **./wrenv.sh -p vxworks-7**
Windows	To install directly on the host: Download for Windows 10 from the following, ensuring you select **Add Python to PATH**: https://www.python.org/downloads/ Or, to use the version included with your VxWorks 7 installation: $ **cd** *installDir* $ **wrenv.exe -p vxworks-7**

5. Create or navigate to your workspace directory; that is, *workspaceDir*.
6. If required, invoke the script using the -h option to see the syntax.

```
$ installDir/sdk-generator/generate_sdk.py -h
```

```
usage: generate_sdk.py [-h]
                       {-vip VIP Project Directory | -l Load SDK configuration}
                       [-sdk SDK Output Directory] [-o SDK Name]
                       [-q QEMU_DIR]
                       [--app Application Type Support]
                       [-f]
                       [-a]
                       [--docs Documentation Generation Level]
                       [-v]
                       [-c]
                       [--add ADD_PATH_LIST [ADD_PATH_LIST ...]]
                       [--exclude [DISABLED_FEATURES [DISABLED_FEATURES ...]]]
                       [--include [ENABLED_FEATURES [ENABLED_FEATURES ...]]]

optional arguments:
  -h, --help            show this help message and exit
  -sdk SDK Output Directory, --sdk SDK Output Directory
                        Path to SDK output directory
  -o SDK Name           Change the default SDK output name
  -q QEMU_DIR, --qemu QEMU_DIR
                        Path to QEMU installation
  --app Application Type Support, --application-type Application Type Support
                        Defaults to producing an SDK that supports both RTPs and DKMs.
                        Choices are: rtp_and_dkm, dkm_only, rtp_only
  -f                    Overwrite the existing SDK output directory
  -a, --artifacts       Produce VSB/VIP build artifacts
  --docs Documentation Generation Level
                        Default is no documentation.
                        Choices are: no_docs, vip_docs
  -v                    Output additional information to the console
  -c, --compress        Compress the SDK into a tarbz/zip format
```

```
  --add ADD_PATH_LIST [ADD_PATH_LIST ...]
                        Add extra folders/files to generated SDK in subfolder /user_content/
                        If a folder is supplied, all subfolders will be included also
  --exclude [DISABLED_FEATURES [DISABLED_FEATURES ...]]
                        Features to disable. By default, python is disabled.
                        Choices are: python, cmake
  --include [ENABLED_FEATURES [ENABLED_FEATURES ...]]
                        Features to enable. By default, cmake is enabled.
                        Choices are: python, cmake

required arguments (choose at least one):
  -vip VIP Project Directory, --vip VIP Project Directory
                     Path to VIP project directory
  -l Load SDK configuration, --load Load SDK configuration
                     Load a pre-defined SDK configuration file
```

Note:

- The --add option is used to include any files that the application developer needs; for example, Python, Boost, or Rust source files.
- The --add, --include, and --exclude options take list arguments as --option x y z. No quotations or commas are needed.
- When not using -o, a JSON file is produced in the current directory (named *<SDK_NAME>* **.json**) alongside every SDK that is generated.

  Use the --load option with such a JSON file to reproduce an SDK with the same arguments.
- The --artifacts option produces a VSB/VIP build artifact (found in the *<SDK_DIR>* **/artifacts** directory), that can be used by the vxbuild.py script to reproduce the original VSB and VIP.

7. Run the generate_sdk.py script to create the SDK.

   With the number of script options available, you can see there are a lot of combinations. Three example are shown here:

   1. Example 1 is an SDK using the minimum set of options. This is just pointing to the VIP. There is no generated reference documentation such as API References, and the default directory structure is used.
   2. Example 2 is an SDK with reference documentation, including QEMU support, using a non-default SDK name, compressed for easier delivery to the application developer.

   Example 1:

   From the *workspaceDir*, using a VIP named **testvip**, based on the X86 itl_generic BSP, with CPU CORE:

   ```
   $ installDir/sdk-generator/generate_sdk.py --vip testvip
   ```

   This generates an SDK contained in a subdirectory; for example, by default, the directory name includes the VIP name, the architecture, the host type, and the date and time the SDK was generated:

   - **WRSDK_VXWORKS-7_TESTVIP_I86_LINUX_2020-06-22_17-49-04**

   Provide this directory (*sdkInstallDir*) and its contents to the application developer to enable them to begin their work.

   Example 2:

   From the *workspaceDir*, using a VIP named **testvip**, based on the X86 itl_generic BSP, with CPU CORE, with generated reference documentation (--docs), including QEMU support (--qemu) using a non-default SDK name (-o), and compressed (-c):

```
$ installDir /sdk-generator/generate_sdk.py --vip testvip --docs vip_docs -o SDK-testvipQemu2 --qem
u ../../Qemu4.1.0/qemu-4.1.0 -c
```

This generates an SDK contained in the **.tar.bz2** file:

- **SDK-testvipQemu2.tar.bz2**

Provide this file to the application developer to enable them to begin their work. The application developer extracts the contents into a preferred *sdkInstallDir* .

8. It is recommended to test the SDK for basic sanity.

   The generated SDK package can be tested with a helper script named gen_sdk_test.py found in *installDir* **/sdk-generator/scripts**. The script tests the following:

   - Builds the examples present within the SDK.
   - If the IP of a running target is also provided, it will run the Hello World RTP and DKM examples on the target.
   - In order to use the IP, boot VxWorks on the target hardware or QEMU. For more details on starting a target with the desired VxWorks image, see:
     VxWorks 7 SDK Boot Guide

Host Type	Instructions
Linux	`$ installDir /sdk-generator/scripts/gen_sdk_test.py -sdk sdkInstallDir -ip 127.0.0.1`
Windows	`$ installDir \sdk-generator\scripts\gen_sdk_test.py -sdk C:/sdkInstallDir -ip 127.0.0.1`

5. TROUBLESHOOTING

This describes currently known issues you may encounter with generating SDKs and some possible solutions.

Problem	Solution
The generate_sdk.py script produces a file not found or No such file or directory when generating the toolkit.	By default, Windows allows at most 260 characters for a file path. To avoid this problem: 1. Try changing the default SDK name using the -o and -sdk options using a shorter SDK path. 2. Alternately, enable long paths by updating the Windows Registry; see: Windows Long Paths
The generate_sdk.py script produces a createProfile failed : profile name already exists message when generating the artifacts.	Rebuild the VSB and VIP projects.